Computers & Graphics 91 (2020) 219–231

Contents lists available at ScienceDirect

Computers & Graphics

journal homepage: www.elsevier.com/locate/cag

Technical Section

Visualization of Nil, Sol, and $\widetilde{SL_2(\mathbb{R})}$ geometries ☆

Tiago Novello [a,*], Vinícius da Silva [b], Luiz Velho [a]

[a] IMPA - VISGRAF Laboratory, Rio de Janeiro, Brazil
[b] Pontifical Catholic University of Rio de Janeiro - PUC-Rio, Department of Informatics, Rio de Janeiro, Brazil

ARTICLE INFO

Article history:
Received 15 April 2020
Revised 3 July 2020
Accepted 28 July 2020
Available online 5 August 2020

Keywords:
Ray tracing
Non-Euclidean geometries

ABSTRACT

In this work, we propose a novel ray tracing model for immersive visualization of Riemannian manifolds. To do this we introduce Riemannian ray tracing, a generalization of the classic Computer Graphics concept. Specifically, our model is capable of interactive real-time VR visualizations of *Nil*, *Sol*, and $\widetilde{SL_2(\mathbb{R})}$, Thurston's most nontrivial geometries. These experiences have the potential to allow insights with impact in physics/cosmology research, education, special effects, and games, among other aspects. The Riemannian ray tracing is implemented using the ray-tracing capabilities of the NVidia RTX platform. We discuss the general algorithm in CPU and show how to map the computations to the RTX pipeline.

1. Introduction

In 2018, the NVIDIA RTX GPUs introduced hardware support for real-time ray tracing algorithms, thus enabling visualization applications with an unprecedented degree of photo-realism. In this paper, we take the challenge of applying the power of the RTX architecture to the exploration of mathematical spaces that feature nontrivial geometry and topology in Virtual Reality (VR).

While the most obvious use of the RTX GPUs is for real-time photo-realistic simulation with applications in entertainment, architecture, design, etc., there are other areas where this power can open up new perspectives not imaginable before. One of these areas is the *Visualization of Mathematics*. In this realm, abstract concepts, such as high dimensional spaces, Non-Euclidean geometries, and non-trivial topologies and manifolds, can be made concrete for immersive exploration.

Using VR and ray tracing it is now possible to create these Mathematical landscapes for interactive visualization, putting the viewer inside them for intuitive understanding. Such experiences have the potential to allow many insights with great impact in research, education, and games, among other aspects.

This work presents the development of new real-time ray tracing methods for immersive views of manifolds locally modeled by Nil, Sol, and $\widetilde{SL_2(\mathbb{R})}$ geometries: Thurston's three most nontrivial geometries. In addition to their mathematical aspects, these spaces have interesting applications in physics [1] and cosmology [2]. Specifically, our **contributions** are:

- The introduction of Riemannian ray tracing, generalizing classic Computer Graphics concepts.
- The proposal of a ray tracing model for immersive visualization of Riemannian manifolds.
- Realtime VR visualization of Nil, Sol, and $SL_2(\mathbb{R})$: the most nontrivial Thurston geometries.

To the best of our knowledge, this is the first project that combines real-time ray tracing and VR for the exploration of those important mathematical spaces. VR is a strong candidate tool to help understand how the geometry of these spaces behaves. In this context, a person can interact with it by changing movement and point-of-view, and evaluating how the space distorts in consequence.

Fig. 1 shows inside views of spaces modeled with the geometry of Nil, $\widetilde{SL_2(\mathbb{R})}$, and Sol, respectively. Expressing their behavior with static images is not trivial. Because of their non-isotropic nature, the spaces looks very different depending on the viewer's point of view and position. On the left, we present a space modeled with the geometry of Nil. We can see one of the properties that characterize Nil in the figure: it is a plane with one line (the edge colored by red and blue) for each point, a (twisted) product of $\mathbb{R}^2$ by $\mathbb{R}$. In the middle, we present a space modeled with $\widetilde{SL_2(\mathbb{R})}$ geometry. Like in Nil, this space is a "product", however, here $\mathbb{R}^2$ is replaced by the hyperbolic plane. The image tries to convey the feeling of being inside that structure. Finally, on the right, we have a space modeled by Sol. This is the least symmetric geometry as one can see in the image [2]. This manifold has variation in two directions

☆ This article was recommended for publication by Markus Hadwiger.

* Corresponding author.
E-mail addresses: tiago.novello@impa.br, tiago.novello90@gmail.com (T. Novello).

https://doi.org/10.1016/j.cag.2020.07.016
0097-8493/

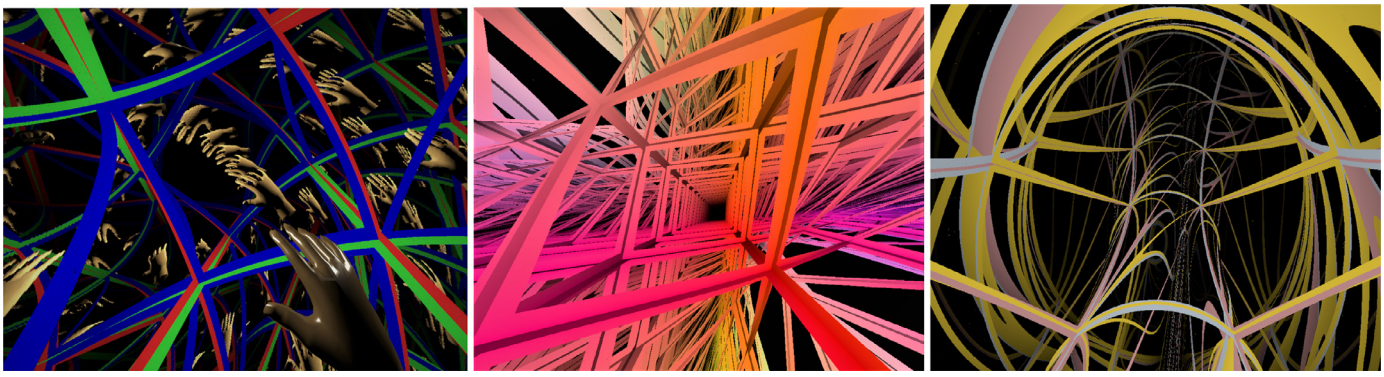


Fig. 1. Immersive visualizations of spaces modeled by Nil, $\widetilde{SL_2(\mathbb{R})}$, and Sol geometries. On the left, we present a space modeled with the geometry of Nil. We can see one of the properties that characterizes Nil in the figure: it is a plane with one straight line (edges in red and green) for each point, a "twisted product" of $\mathbb{R}^2$ by $\mathbb{R}$. In the middle, we present a space modeled with $\widetilde{SL_2(\mathbb{R})}$ geometry. As Nil, this space is a "twisted product", however, here the $\mathbb{R}^2$ is replaced by the hyperbolic plane. Finally, on the right, we have a space modeled by Sol. This is the least symmetric geometry as one can see in the illustration: space has variation in two directions unlike Nil and $\widetilde{SL_2(\mathbb{R})}$.

along a line [3]. In Section 7 we describe in detail the structures that the images are showing.

Another option to explore those spaces is by video, an intermediary media between image and VR. It is not as controllable as VR, but it is possible to see the behavior of the spaces to some degree. We provided videos of the VR sessions as supplementary material and encourage the reader to also look at them.

The paper is structured as follows: Section 2 reviews previous and related works and motivates ray tracing in Riemannian geometry; Section 3 discusses the core concepts used in this paper; Section 4 introduces concepts from Riemannian geometry and Lie groups; Section 5 discusses the possibilities of immersive visualization of 3-manifolds and introduces Riemannian shading and illumination, which enable Riemannian ray tracing in such spaces; Section 6 introduces the ray tracing algorithm to compute a Riemannian shading of a given 3-manifold; Section 7 presents ray-traced inside views of Nil, Sol, and $\widetilde{SL_2(\mathbb{R})}$ spaces, which are defined in Appendix A, Appendix B, and Appendix C, respectively; Section 8 shows how to map the ray tracing algorithm to the RTX pipeline and present the details of the GPU implementation; Section 9 discusses more implementation details and evaluates empirical data; finally, Section 10 gives concluding remarks.

2. Related work

The main effort for mathematics visualization, particularly of Non-Euclidean spaces, took place at the Geometry Center from 1994 to 1998. This initiative, under the leadership of William Thurston, resulted in a scientific program to study and disseminate modern geometry using interactive visualization.

For this purpose, a platform called *Geomview* [4] was developed. The software was based on OpenGL and supported interactive viewing in Euclidean, spherical, and hyperbolic spaces. Weeks [5] proposed real-time visualization of these classical geometries, and in [6], he extended it to the product geometries.

Researchers at the Geometry Center, already at that time realized the potential of Virtual Reality for providing insights into the world of curved spaces. They created simple VR installations to allow the user, not only to have a glimpse at the visual landscape inside a 3-manifold, but also to experience the sensation of being immersed in such an environment. Two of their projects are Mathenautics [7] and Alice [8]. Recently, Hart et al. [9] presented a sophisticated immersive exploration of such curved spaces using VR, and Weeks [10,11] improved it by presenting a framework that allows the development of games.

Another initiative in the direction of using VR for mathematics visualization was JReality [12], a Java-based 3D scene graph package designed for mathematical visualization at TU Berlin.

The early work on interactive visualization of Non-Euclidean spaces, reported above, was based on the traditional OpenGL rasterization pipeline [5]. Therefore, the rendering algorithms employed a scene-based architecture. There are two problems with such an approach. First, the scene must be replicated to "unroll" spaces with complicated topology (it will be clarified along the text). Second, to rasterize a scene it is necessary to project objects in the scene based on camera position. This is a hard problem in manifolds with non-trivial geometry, where classic perspective projection cannot be applied. The ray-tracing pipeline allows us to overcome such difficulties operating intrinsically in the manifold geometry and topology.

The first work to propose the use of an *Image-Based* architecture for visualization of Non-Euclidean spaces using GPUs was from Berger et al. [13]. Their rendering algorithm exploited programmable compute shaders and CUDA to implement ray tracing on the GPU. However, their work does not provide real-time immersive and interactive visualization, and it was restricted to Euclidean and hyperbolic manifolds. Rays in such spaces are modeled by straight lines which allow the use of classical algorithms from computational geometry.

In [14,15], Velho et. al proposed a real-time immersive and interactive visualization of Euclidean, Hyperbolic, and Spherical spaces. This work provides real-time immersive visualization of manifolds modeled by Nil, Sol, and $\widetilde{SL_2(\mathbb{R})}$ geometries by proposing a ray tracing model for Riemannian manifolds; the rays in such spaces can not be modeled by straights lines.

The first attempt to visualize Nil, Sol, and $\widetilde{SL_2(\mathbb{R})}$ in real-time (using VR) appeared in 2019 [16]. The visual impact inspired many explorations of such spaces [17–19].

3. Core concepts

To visualize the examples of Riemannian manifolds in this work, it is important to understand two related core concepts: geodesics and fundamental domains. They are related to the manifold geometry and topology, respectively. This section gives an informal intuition for both, preparing the reader for details in forthcoming sections.

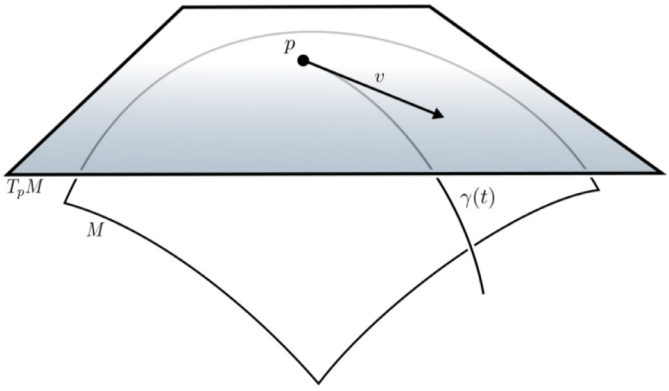


Fig. 2. We illustrate the manifold M and the tangent space T_pM at the point p as two-dimensional objects. The geodesic $\gamma(t)$ starts at p in the direction v.

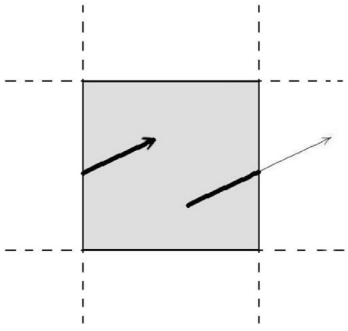


Fig. 3. 2D torus. Each edge of the square (fundamental domain) is identified with the opposing one. Because of that, everything inside the square is replicated in the entire $\mathbb{R}^2$ (covering space). Image from [13].

3.1. Geodesics and fundamental domain

Geodesics are the equivalent to straight lines in the Euclidean space: paths locally minimizing lengths. They are a consequence of the manifold geometry, causing the perception of deformation that a viewer has when exploring the space. Section 4.3 gives the precise definition. Fig. 2 shows a geodesic $\gamma(t)$ in a Riemannian manifold M. It passes through a point p in a tangent direction v lying in the tangent space T_pM: the set of all possible directions for geodesics passing through p. Note the parametric notation for the geodesic, such as a line in Euclidean space.

The fundamental domain is a "polyhedron" such that a special face identification describes the topology of the manifold. This concept is defined in Section 3.3. In dimension 2, for example, a 2D Torus is obtained by identifying the opposing edges of a square, which is the fundamental domain. Embedding this square in $\mathbb{R}^2$ and "interpreting" the identifications as translations we get a tessellation of $\mathbb{R}^2$ by squares, which is called the covering space of the torus. Fig. 3 shows the 2D Torus in the covering space.

3.2. Non-Euclidean Ray tracing

Ray tracing [20] consists of following the path traveled by light in the space to evaluate the contribution of light sources to pixels in an image. The classic algorithm assumes linear paths in Euclidean space.

We propose a more general approach by abstracting how light travels through space. Instead of following linear rays, the former now follows geodesics imposed by the latter. In other words, it suffices to know the geodesics of a space to ray trace it. Note that the Euclidean space is a particular case in this model, where the geodesics are lines.

A Riemannian manifold can be visualized using ray tracing because its geodesics propagate on the fundamental domain deforming and tessellating the manifold covering space. Fig. 3 shows a ray being that exits and enters the fundamental domain as it travels the covering space.

Although this is a simple conceptual change to the classic ray tracing algorithm, it is powerful. To prove our point, we show examples of how to mathematically model the geodesics in detail for non-trivial spaces, resulting in interesting visualizations and deforming effects. Later on, we provide implementation details to describe how to compute the algorithm in real-time.

3.3. Riemannian manifolds and Lie groups

To motivate the mathematical definitions used in this work we remember that it deals with an immersive visualization of curved spaces — rays may not be straight lines — using ray tracing. Then the space must satisfy at least three properties:

- Being locally similar to a Euclidean space, a *manifold*. This allows us to model the viewer/scene inside the space.
- To simulate effects produced between the lights and the scene we need the *tangent space* with a *scalar product* at each point, a *Riemannian metric*.
- The *ray* leaving a point in any direction; the *geodesic*. The ray-object intersections are also required.

Riemannian manifolds carry the above properties. We are interested in a specific class of them — the *model geometries*. These spaces have simple topology (*simply connected* and *complete*) and each pair of points admits neighborhoods with similar geometry (*homogeneous spaces*). In dimension two, for example, there are exactly three models: Euclidean, hyperbolic, and spherical. In dimension three, there are five more model geometries. We explore three of them: *Nil*, *Sol*, and $\widetilde{SL_2(\mathbb{R})}$ — the three most nontrivial Thurston geometries [21]. Those geometries are examples of *Lie groups*: manifolds that admit a group structure. The work [14] approached *Euclidean*, *hyperbolic*, and *spherical* spaces. The remaining geometries are $\mathbb{S}^2 \times \mathbb{R}$ and $\mathbb{H}^2 \times \mathbb{R}$ with the product metric. Since they are simpler and model fewer manifolds, we opt to omit them here.

Model geometries date back to the famous *Thurston geometrization conjecture*, solved in 2003 by Grigori Perelman. This states that every compact three-dimensional manifold decomposes into pieces with the geometry modeled by Thurston geometries. A manifold N has its geometry modeled by a model geometry M if it can be expressed as a quotient $N = M/\Gamma$, where Γ is a discrete group of isometries acting on M. The manifold N inherits the metric of M — and it is called a *geometric manifold*. The *fundamental domain* of N is the region in M containing a point for each orbit (the images of a single point under the action of Γ). Great texts on this subject are [3,21].

4. Riemannian geometry

Riemannian geometry [22] was developed from the study of surfaces in $\mathbb{R}^3$. There is a natural way to measure lengths of vectors in a tangent plane of a surface $S \subset \mathbb{R}^3$ because it inherits the scalar product of $\mathbb{R}^3$. This implies a definition of distance between two points in S. Curves in S that minimize distance are called *geodesics*. From now on we use the terms *geodesic* and *ray* interchangeably.

Our goal in this section is to present the definitions for Riemannian manifolds, their geodesics, and Lie groups as examples. Our ray tracing model will operate in this setting. We follow the exposition in Carmo [22].

4.1. Manifold

A *3-manifold* M is a topological space which is locally identical to the Euclidean space $\mathbb{R}^3$. More precisely, there is a neighborhood of every point in M mapped diffeomorphically to the open ball of $\mathbb{R}^3$. These maps are called *charts* of M. The change of charts between two neighborhoods in M must be a diffeomorphism. Thus, informally, the manifold definition generalizes the concept of Euclidean space. Examples of 3-manifolds include classical geometries: Euclidean, hyperbolic, and spherical spaces, as well as non-classic geometries: Nil, Sol, and $SL_2(\mathbb{R})$ (see Appendix A, Appendix B, and Appendix C). Fig. 2 shows a schematic view of a 2-manifold.

Tangent space: Let p be a point in the 3-manifold M, and $X(x_1, x_2, x_3)$ be a chart (parameterization) of a neighborhood of p. Then the *tangent space* T_pM in p is the vector space generated by the vectors $\{\frac{\partial}{\partial x_i}(p)\}$ of the coordinates curves at p (see Fig. 2). As T_pM admits a linear vector space structure, we can define a scalar product on each tangent space of M. If such procedure is "smooth" the result is a metric on M.

4.2. Metric

A *Riemannian metric* in a manifold M is a map that attributes to each point p in M a scalar product $\langle\cdot,\ \cdot\rangle_p$ in the tangent space T_pM, such that in coordinates, $X(x_1, x_2, x_3) = p$, the function $g_{ij}(x_1, x_2, x_3) := \langle \frac{\partial}{\partial x_i}, \frac{\partial}{\partial x_j}\rangle_p$ is smooth. This function does not depend on the coordinate system. We may use g instead of $\langle\cdot,\ \cdot\rangle$.

Let p be a point in M, and u, v be tangent vectors of T_pM. Expressing u and v in terms of the basis associated to a chart $X(x_1, x_2, x_3)$, that is, $u = \sum u_i \frac{\partial}{\partial x_i}(p)$ and $v = \sum v_i \frac{\partial}{\partial x_i}(p)$, we obtain the scalar product between u and v:

$$\langle u, v\rangle_p = \sum_{i,j=1}^{3} u_i v_j \left\langle \frac{\partial}{\partial x_i}, \frac{\partial}{\partial x_j}\right\rangle (p) \tag{1}$$

The metric g on M is completely determined by the matrix $[g_{ij}] := [\langle \frac{\partial}{\partial x_i}, \frac{\partial}{\partial x_j}\rangle(p)]$. A *Riemannian manifold* is a pair (M, g), where M is a manifold and g is a Riemannian metric on M.

4.3. Geodesics

We now investigate the geodesics (rays) in the Riemannian manifold (M, g). In Euclidean space, these curves have null acceleration (second derivative). More precisely, a curve $\gamma(t)$ in Euclidean space is a geodesic iff $\gamma''(t) = 0$, in other words, a straight line. The analogous concept of "acceleration" in (M, g) geometry is defined using the concept of *covariant derivative*. This is a way to calculate the derivative of tangent fields along curves in manifolds.

Let $\gamma(t) = (x_1(t), x_2(t), x_3(t))$ be a curve in (M, g), the *covariant derivative* of $\gamma'(t)$ is defined as [22]:

$$\frac{D}{dt}\big(\gamma'(t)\big) = \sum_{k=1}^{3}\left(x_k''(t) + \sum_{i,j=1}^{3}\Gamma_{ij}^k x_i' x_j'\right)\frac{\partial}{\partial x^k}.$$

Γ_{ij}^k are the *Christoffel symbols* of the unique *Riemannian connection* associated with (M, g). This connection is obtained using the *Levi–Civita theorem* [22]. These symbols correlate coordinate derivatives with covariance derivatives and measure how the tangent space basis is modified as the coordinate system moves along the geodesic. Finally, $\gamma(t)$ is a *geodesic* (*ray*) if $\frac{D}{dt}\big(\gamma'(t)\big) = 0$, in other words

$$x_k'' + \sum_{i,j=1}^{3}\Gamma_{ij}^k x_i' x_j' = 0,\quad k = 1, 2, 3. \tag{2}$$

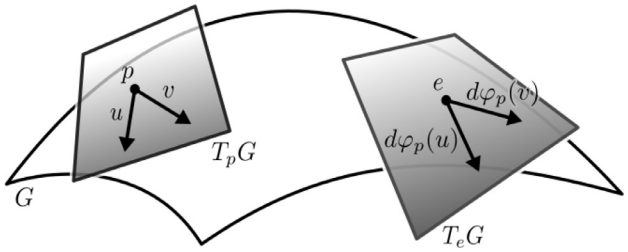


Fig. 4. Vectors belonging to the tangent space T_pG are transported to the tangent space at origin by the differential of φ.

This definition differs from the Euclidean case by the addition of the term $\sum \Gamma_{ij}^k x_i' x_j'$. Fig. 2 illustrates a geodesic.

In order to solve Eq. (2), which has order two, we add a new variable being its first derivative. Specifically, to compute a ray $\gamma(t) = (x_1(t), x_2(t), x_3(t))$ we consider the *tangent bundle* TM of M. This space consists of all pairs of points and tangent directions of M, that is $\{(p, v) | p \in M,\ v \in T_pM\}$. In this space the ray $\gamma(t)$ can be expressed by $t \to (\gamma(t), \gamma'(t)) = (x_1(t), x_2(t), x_3(t), y_1(t), y_2(t), y_3(t))$. The numbers y_k are the coordinates of the tangent vectors. Rewriting Eq. (2):

$$\begin{cases} x_k' = y_k \\ y_k' = -\sum_{i,j=1}^{3}\Gamma_{ij}^k y_i y_j, \quad k = 1, 2, 3. \end{cases} \tag{3}$$

Eq. (3) define a vector field on TM which is called *geodesic flow* of (M, g). Then, to compute the geodesic flow in a Riemannian manifold we need the Christoffel symbols at each point, which are known [22] to be given by the formula:

$$\Gamma_{ij}^m = \frac{1}{2}\sum_{k=1}^{3}\left(\frac{\partial}{\partial x_i}g_{jk} + \frac{\partial}{\partial x_j}g_{ki} - \frac{\partial}{\partial x_k}g_{ij}\right)g^{km}. \tag{4}$$

Where $[g^{ij}]$ is the inverse matrix of $[g_{ij}]$.

A rich and powerful class of Riemannian manifolds, the *Lie groups*, have specific properties useful for our ray tracing model. Nil, Sol, and $\widetilde{SL_2(\mathbb{R})}$ geometries are special examples of them and they will be our focus from now on.

4.4. Lie groups

A *Lie group* is a manifold G which is also a group and its operations $(p, q) \to p \cdot q$ and $p \to p^{-1}$ are smooth. Thus its *left multiplication* L_p by $p \in G$, given by $L_p(q) = p \cdot q$, is smooth.

The classical way to define a metric in a Lie group G is by fixing a scalar product $\langle\cdot,\ \cdot\rangle_e$ in the tangent space at the identity element e of G. Then extending it by left multiplication. Specifically, we define a Riemannian metric in M as follows:

$$\langle u, v\rangle_p = \langle d\varphi_p(u), d\varphi_p(v)\rangle_e,\quad p \in M,\ u, v \in T_pG. \tag{5}$$

To simplify notation we considered $\varphi := L_{p^{-1}}$. Fig. 4 illustrates the transport of vectors from T_pG to T_eG.

Appendix A, Appendix B, and Appendix C present three important examples of Lie groups endowed with a natural Riemannian geometry: Nil, Sol, and $\widetilde{SL_2(\mathbb{R})}$ geometries. We propose an inside visualization of spaces with non-trivial topology that have the geometry modeled by such geometries. Before, a general definition of shading is presented for Riemannian geometries (Section 5).

5. Visualization of Riemannian manifolds

There are two obstacles when visualizing a manifold. First, it is necessary to compute *rays*, since light propagates along them. As

we saw in the previous section, the *geodesic flow* is perfect for that, providing a ray for each point and tangent direction (see Eq. (3)). The second obstacle is the dimension; our eyes only see up to dimension three.

5.1. Visualization approaches

Two-dimensional manifolds can be visualized using two approaches: *extrinsic* or *intrinsic*. In the extrinsic setting, we consider the manifold M embedded in a higher-dimensional space, typically $\mathbb{R}^3$, i.e., $M \subset \mathbb{R}^3$. In this way, M can be seen from the *outside* using a virtual camera. In the intrinsic setting, we consider the manifold as an abstract independent space. Therefore, M can only be seen from the *inside*. It is also possible to visualize the *universal covering*, i.e., the tessellation produced by the group action through views of the covering space itself. In any case, these visualizations can be based on classical CG algorithms: *rasterization* or *ray tracing*.

The problem of visualizing 3-manifolds is harder. In 1998, Thurston published *How to see 3-manifolds* [23], discussing ways to "see" a 3-manifold using our spatial imagination and computer aid. Many tools in Riemannian 3-manifold are inspired in human mind geometrical instincts. Thus, the human mind is trained to understand the kinds of geometry that are needed to model certain 3-manifolds.

Since higher-dimensional manifolds "cannot" be used to visualize 3-manifolds, we take an immersive approach based on a *ray tracing* algorithm. It follows the ideas from [13]. Rasterization is not appropriate for this scenario because perspective projection in Non-Euclidean spaces is nontrivial. On the other hand, a scene embedded in a 3-manifold can be ray traced: given a point (eye) and a direction (pixel), we trace a geodesic (ray). When it hits an object we compute its *shading*.

5.2. Riemannian ray tracing

In computer graphics, *Shading* is the process of assigning a color to a pixel. Classic approaches to perform such tasks are not suited for Riemannian manifolds. Thus, we propose a more general definition for ray tracing Riemannian geometry.

Consider a point p (the eye) in a 3-manifold M, and also consider the set of directions within the observer's field of view V_p (the view frustum). We compute a color by tracing rays in the directions associated with image pixels. Specifically, p is the origin of T_pM, and for each direction $v \in V_p$ we attribute a color c by launching a ray $\gamma(t)$ from p in the direction v. Each time γ intersects a visible object at a point q we define an RGB color. Therefore, we have $c : V_p \to \mathcal{C}$, where $\mathcal{C}$ is a color space. We call this procedure *Riemannian shading*. Fig. 5 presents a schematic view of the above procedure in dimension two.

Let q be a point in an embedded surface $S \subset M$. The *Riemannian illumination* of q comes from direct geodesics connecting q to the light sources and indirect geodesics connecting other Riemannian-illuminated points to q. The *radiant intensity* at q can be modeled using the *Lambertian reflectance*, which depends on the inner product, or more generally from a BRDF. Thus, it is natural to adapt this model to the Riemannian geometry by replacing the standard inner product with the Riemannian metric of the underlying 3-manifold. For a given point p in M we compute the Riemannian shading $c : V_p \to \mathcal{C}$ using ray tracing and the Riemannian illumination.

In classic approaches, ray tracing [20] approximates physical illumination. Our ray tracing model for Riemannian 3-manifolds can be also used to compute a Riemannian shading function for local or global illumination.

This work, however, uses pseudo-color based either on properties of the space, such as curvature, or attributes of the objects, such as surface normal, to define the Riemannian shading function. In future works, we plan to compute the illumination of Nil, Sol, and $\widetilde{SL_2(\mathbb{R})}$, for that, it would be necessary to calculate direct geodesics, which could require a nontrivial effort.

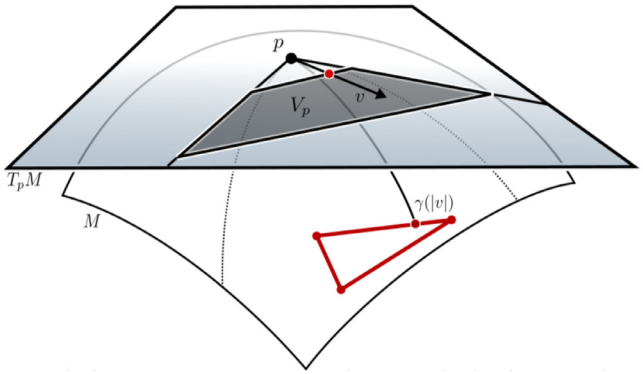


Fig. 5. Shading in a Riemannian manifold M. Let p be the observer and V_p be the view frustum (gray region) in the tangent space T_pM. We launch a ray γ towards each vector $v \in V_p$. If γ hits a visible object (red triangle) in $\gamma(|v|)$ we define a RGB color (red) for the corresponding point in the near plane of V_p.

6. Ray tracing in Riemannian manifolds

We provide an algorithm to compute a Riemannian shading that synthesizes inside views of Non-Euclidean spaces. This generalizes the classical *ray tracing*. A similar version focused on visualizing classical geometry can be found in [14]. We think that presenting it would make the work self-contained.

We focus on a geometric manifold M/Γ since the rays in the model geometry M are well-behaved and its topology is given by Γ. Our algorithm computes a Riemannian shading of the visible surfaces. We discuss the basic principles of ray tracing in these manifolds, as well as the general algorithm in CPU. Section 8 shows the map of the computation to the RTX pipeline.

6.1. Overview of the method

Ray tracing is a natural method to visualize 3-manifolds. It is necessary to adapt the traditional ray tracing to consider the geometry/topology of the space. The first aspect of this task is to simulate the ray path as it travels inside the space. The second aspect amounts to a shading procedure, which computes the illumination and evaluates the light scattered from the environment in the ray direction. Because of the non-trivial topology, the ray path is updated as it exits the fundamental domain.

6.2. Algorithm in CPU

Algorithm 1 presents the basic Ray tracing procedure for a geometric manifold M/Γ. The rays are generated from the observer's point of view (lines 1-4), intersected with visible objects (line 6) and if there is a hit (line 6), shading is done (line 8). Because of practical computational reasons, we cannot continue the ray path indefinitely and set the maximum recursion level (line 13).

These steps are common to any ray tracing algorithm, including the traditional one for the Euclidean space. To trace rays in a geometric manifold M/Γ, we need extra steps to guide the ray as it iterates in the fundamental domain (lines 9, 10, and 11).

When Γ is the empty set, the ray tracing algorithm coincides with the classical (line 9). In this case, the trace ray function (line 4) will trace curved rays. The most important and critical step is the group action (line 11), which depends on the geometry/topology of the manifold. It is specific for each space type.

6.3. Ray marching

To launch a ray we need to solve the geodesic flow (Eq. (3)) and subsequently compute the intersection of the given ray with the scene objects. In general, our rays will be *curved rays* that fall into two cases: (i) for a few manifolds the flow admits a closed solution resulting in a parametric curve $\gamma(t)$; (ii) for most manifolds, however, the geodesic flow does not have a closed-form solution and we have resort to numerical integration methods.

In case (i) we generate a polygonal curve by discretizing the parameter t, such that $\gamma(t) = \{p_i\}$. In case (ii) we use Euler's method for integration in the parameterization image, since there we use the Euclidean metric.

The Euler's numerical integration method approximates a ray γ starting at p in the direction v by a polygonal $\{p_i\}$, where:

$$\begin{cases} p_{i+1} = p_i + h \cdot \widetilde{\gamma}'(0) \\ v_{i+1} = v_i + h \cdot \widetilde{\gamma}''(0) \end{cases} \tag{6}$$

where h is the integration step and $\widetilde{\gamma}(t)$ is the ray satisfying $\widetilde{\gamma}(0) = p_i$ and $\widetilde{\gamma}'(0) = v_i$. We use Eq. (3) to compute $\widetilde{\gamma}''(0)$.

In any case, the polygonal curve $\{p_i\}$ is used to ray trace a scene in (M, g). When $h \to \epsilon$ the scene is rendered with more accuracy. To compute the intersection between a ray and the scene objects we test the intersection between each segment given by the polygonal approximation given by the above computation (i.e., ray marching).

For now, we are testing the intersection with each segment sequentially. In future works, we intend to use many segments of the polygonal $\{p_i\}$ to increase the GPU occupancy and optimize the intersection computations.

We could use the RungeKutta method to approximate solutions of the geodesic flow. Instead, we take the Euler method since it uses fewer computations giving GPU performance.

7. Visualization of Nil, Sol and $\widetilde{SL_2}(\mathbb{R})$

This work focuses on applying Riemannian ray tracing to render inside views of the most non-trivial Thurston Geometries: Nil, Sol, and $\widetilde{SL_2}(\mathbb{R})$. These Riemannian manifolds are fundamental in the Geometrization conjecture.

Nil, Sol, and $\widetilde{SL_2}(\mathbb{R})$ are examples of *non-isotropic* spaces. That is, their geometries are not the same in all directions because local two-dimensional slides admit different curvatures. As a consequence, their geodesics are not (in general) straight lines in the Euclidean sense, only a few exceptions are, for example, the geodesics in Nil geometry towards the z direction (use the geodesic formula in Appendix A.2). Another property is that these spaces are Lie groups: a class of group admitting manifold structures. The geodesics in Lie groups can be computed at the group identity and then translated to every element.

In Appendix A, Appendix B, and Appendix C, we provide the formal definitions of Nil, Sol, and $\widetilde{SL_2}(\mathbb{R})$ spaces. For the Nil space, we have the analytical solution of the geodesic flow [24], i.e., case (i) of Section 6.3. For the Sol and $\widetilde{SL_2}(\mathbb{R})$ spaces we approximate the geodesics by numerical integration of the flow ODE's, i.e., case (ii) of Section 6.3.

To visualize Nil, Sol, and $\widetilde{SL_2}(\mathbb{R})$ geometries we consider manifolds modeled by them. For Nil and Sol, we choose compact manifolds created by quotienting these spaces by "simple" discrete groups: "translations" in the direction of axis x, y, and z. We use the edges of the respective fundamental domains to visualize the structure of Nil and Sol. The result is a tiling produced by the fundamental domain edges. For $\widetilde{SL_2}(\mathbb{R})$ we take a different approach, we visualize the well-known *special linear group* $SL_2(\mathbb{R})$ which has the geometry modeled by $\widetilde{SL_2}(\mathbb{R})$. To explore the distortions of such a manifold, we consider rendering a grid defined in the domain of a parameterization of $SL_2(\mathbb{R})$ by (x, y, z) coordinates (see Appendix C).

However, expressing the structures of those spaces with static images is not trivial. We recommend the use of VR for better understanding them, due to their non-isotropic properties. Being inside allows a person to better explore and control variations in point-of-view and position, and understand how they change visualization. An intermediate solution is to explore them through a video, which we are making available as supplementary material. In the figures, we will illustrate several points of view inside these geometries commenting on how the underlying structures vary.

Algorithm 1: Ray tracing a Riemannian manifold M/Γ

```
for each pixel σ ∈ I do
    Let p := 0 and v be the direction associated to σ;
    Let Δ be the fundamental domain polyhedron;
    Trace a ray γ from (p, v) inside Δ;
    repeat
        Find closest intersection γ(t) with objects O in Δ;
        if γ(t) ≠ ∅ then
            Shade pixel break;
        else if Γ ≠ ∅ then
            Find intersection of γ with faces of Δ ;
            Compute the new ray (p', v'), and ++i;
        break;
    until i ≤ maxlevel;
```

7.1. Visualizing Nil space

To visualize the Nil space, we consider a compact manifold $M = Nil/\Gamma$ with geometry modeled by *Nil*. The discrete group Γ acts on *Nil*. Putting the view inside such space we obtain images of it tessellated by the fundamental domain of M.

Let Γ be the discrete group generated by the "translations" in the direction of axis x, y, and z: $\Phi_1(p) = (x+1, y, y+z)$, $\Phi_2(p) = (x, y+1, z)$, and $\Phi_3(p) = (x, y, z+1)$. The manifold $M = Nil/\Gamma$ inherits the geometry of the Nil space. Moreover, it provides a 2-torus for each fixed x, resulting in M admitting a foliation by torus. The unit cube is the fundamental domain.

We set the scene inside the cube and ray trace it using the ray formula defined in Appendix A. Each time a ray intersects a cube face we update it using Γ. Specifically, to visualize the scene objects we define the shading $c_{Nil} : V_p \to \mathcal{C}$, where V_p is the observer's field of view in the tangent space at the camera position. c_{Nil} is defined using *Nil* rays and metric (see Appendix A), and definitions discussed in Section 5.2.

Fig. 6 gives four points of view inside of $M = Nil/\Gamma$. The scene is composed of thickening in 2D of the boundary of the fundamental domain faces. Opposite faces receive the same shading. Red, green, and blue for the faces parallel to the axis, x, y, and z, respectively. In the images, the lines shaded with red and green are extended to the infinite as straight lines. These lines are geodesics of *Nil* and are perpendicular to the plane yz; lines with such property foliate the space *Nil*.

In the top left image, the camera points towards z-axes, where the gluing of the cube faces are trivial (like in the 3D torus). This gives copies of the cube by left translation along z-axes. Looking towards the direction of the green faces would provide a similar result, thus we avoid this perspective. In top right and bottom left, the camera points toward x-axes, in both senses. The identification of the cube faces in this direction if nontrivial, producing this

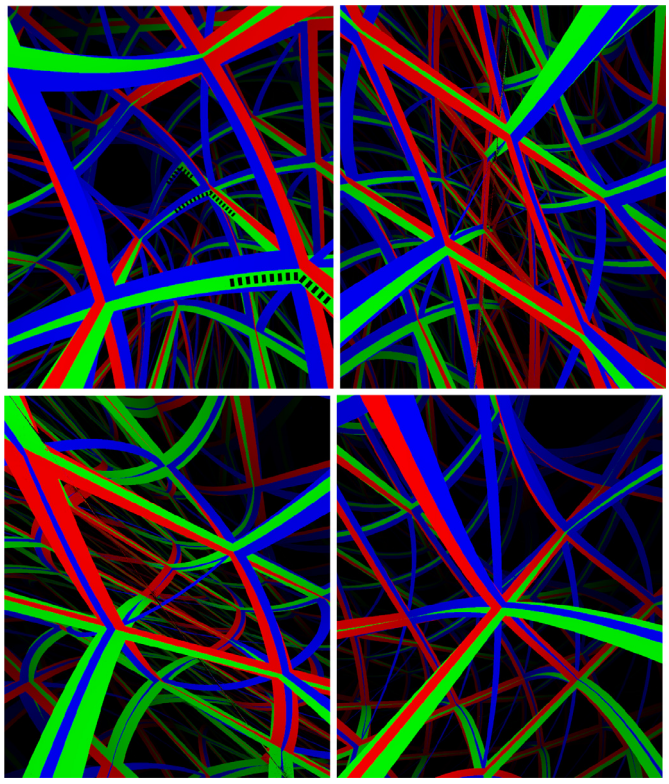


Fig. 6. Inside views of a Nil manifold. The cubes edges compose the scene. The face pairing produces a tessellation of *Nil* by the cubes. In the top left, the camera point towards z-axes, where the group transformation is trivial (like in the 3D torus). Thus we see translated copies of the cube distorted by the Nil geometry. To highlight the distortions we mark (black dotted line) the corner of the cube's bottom face (in green): as the fundamental domain is translated the green face receives an anticlockwise torsion. In the top right and bottom left, the observer looks toward x-axes, in both senses respectively. The identification of the cube faces in this direction if nontrivial producing this tessellation of Nil. In the bottom right, we set the camera towards the cube diagonal. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

tessellation of Nil. Finally, in the bottom right a view towards the diagonal of the cube.

Ray-plane intersection. Let $\beta(t)$ be a ray, such that $\beta(0) = p$ and $\beta'(0) = v$. In Appendix A.2, we see that $\beta(t) = (p_x + x(t), p_y + y(t), p_z + z(t) + p_x y(t))$; $(x(t), y(t), z(t))$ is a ray leaving the origin.

We compute the intersection between $\beta(t)$ and the planes $x = c$ and $y = c$, where c is a constant. For the planes $x = c$, we have to solve $x(t) + p_x = c$, which is equivalent to $\frac{c}{w}(\sin(wt + \alpha) - \sin(\alpha)) + p_x = c$. After some calculation, we obtain

$$t = \frac{\arcsin((c - p_x)\frac{w}{c} + \sin(\alpha)) - \alpha}{w}.$$

In an analogous way we compute the intersection between $\beta(t)$ and the plane $y = c$. The parameter is

$$t = \frac{\arccos((c - p_y)\frac{-w}{c} + \cos(\alpha)) - \alpha}{w}.$$

We could not compute (yet) the intersection of $\beta(t)$ and the plane $z = c$, giving by the solution of $p_z + z(t) + p_x y(t) = c$. We avoid this problem by computing the intersection of rays and the fundamental domain faces using *binary search*. This is reachable because we have the analytic expression of the geodesics of *Nil*. This results in better rendering performance in comparison with the other geometries (Sol and $\widetilde{SL_2(\mathbb{R})}$), which do not have analytic expressions for their geodesics.

7.2. Visualizing Sol space

To visualize *Sol* we consider a compact manifold $M = Sol/\Gamma$ with geometry modeled by *Sol*, where Γ is a discrete group acting on *Sol*. Putting the viewer inside such space we obtain images of Sol tessellated by the fundamental domain of M.

Let Γ be the discrete group spanned by the translations along x- and y-axes, the maps $\Phi_1(p) = (x + 1, y, z)$, $\Phi_2(p) = (x, y + 1, z)$, and $\Phi_3(p) = (x \cdot e^{-2\ln\phi}, y \cdot e^{2\ln\phi}, z + 2\ln\phi)$; where $p = (x, y, z)$ and ϕ is the golden ratio number. The manifold $M = Sol/\Gamma$ inherits the geometry of *Sol*, and for each fixed z_0, it provides a 2D torus which is the quotient of $\mathbb{R}^2 \times \{z_0\}$ by the discrete group generated by Φ_1 and Φ_2. Thus M is foliated by a 2D torus. The parallelepiped $D \times [0, 2\ln\phi)$ is the fundamental domain of M; where D is the unit square.

We explain the role of the element $\Phi_3(p) = (x \cdot e^{-2\ln\phi}, y \cdot e^{2\ln\phi}, z + 2\ln\phi)$ in the discrete group Γ. It is an automorphism between $\mathbb{R}^2 \times \{0\}$ and $\mathbb{R}^2 \times \{2\ln\phi\}$. Observe that the numbers $-2\ln\phi$ and $2\ln\phi$ are the eigenvalues of the matrix $A = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}$, which is an automorphism of $\mathbb{R}^2$ preserving the lattice $\mathbb{Z}^2 \subset \mathbb{R}^2$. Thus A induces a homeomorphism of the 2D torus. Lining up the (x, y)-coordinates of Sol with the eigenvectors of A, we get an identification of A with Φ_3. This procedure to create the Sol manifold M seems to be naóve, however, it can be extended for every automorphism of $\mathbb{R}^2$ preserving the lattice $\mathbb{Z}^2$. For details see Example 3.8.9 in [25] and [18].

We set the scene inside the parallelepiped and ray trace it using Sol rays. Each time a ray hits a parallelepiped face it is updated by the discrete group Γ. As in the Nil section, we define a Riemannian shading for Sol and create the scene by thickening the boundary of the fundamental domain faces and opposite faces receive the same shading. Fig. 7 provides four points of view of $M = Sol/\Gamma$. We look toward the fundamental domain faces and vertex to explore the non-isotropic nature of Sol which is the most non-symmetric geometry. In the top left and right, the camera points towards z-axes, in both senses, respectively. Here the identification of the parallelepiped faces is nontrivial, producing together with its geometry, this tessellation of Sol. The space looks compressed horizontally and vertically, depending on the sense. These regions are highlighted with green dotted lines in the figure. In the bottom left, the camera points in the direction of x-axes, the identification of the cube faces in this direction is trivial (like in the 3D torus). This gives the impression of copies of the fundamental domain very distorted by Sol geometry, surrounding the vertically compressed region. Looking towards y-axis is similar, however, the copies surround the horizontal compressed region, see the right side of Fig. 1. Finally, on the bottom right side, a view towards the diagonal of the fundamental domain. The non-symmetry of Sol is remarkable.

7.3. Visualizing $\widetilde{SL_2(\mathbb{R})}$ space

Here we are not visualizing a manifold generated by taking quotients of $\widetilde{SL_2(\mathbb{R})}$ by discrete groups. Instead, we focus on the visualization of the special linear group $SL_2(\mathbb{R})$ which has the geometry modeled by $\widetilde{SL_2(\mathbb{R})}$; see Appendix C for details. This manifold corresponds to the set of 2×2 matrices with determinant 1. This space is well-known due to its variety of properties, thus its visualization is interesting.

As in the Nil and Sol spaces, we define a Riemannian shading for $SL_2(\mathbb{R})$ and we illustrate it in Fig. 8. This image refers to an inside view of a grid defined in the domain of a parameterization of $SL_2(\mathbb{R})$ by (x, y, z) coordinates. The parameterization distorts the grid following the geometry of $SL_2(\mathbb{R})$. We choose empirically the

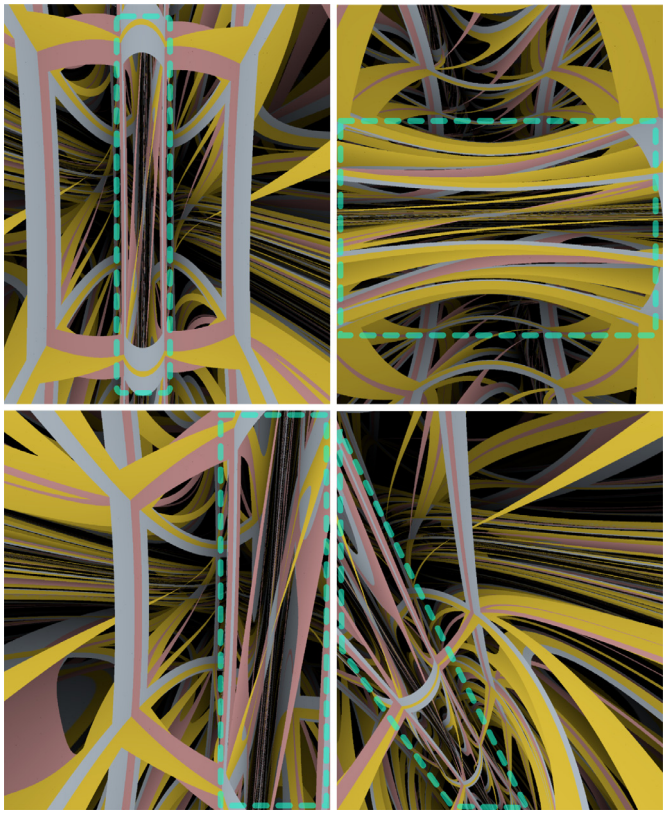


Fig. 7. Immersive visualizations of Sol manifold. The parallelepiped's edges compose the scene. The face pairing makes the rays that leave a face return from its opposite face, tessellating *Sol*. The borders of each face are shaded with different colors to provide a better understanding of the space structure. In the top, the camera points towards direction z, in both senses, respectively. In this direction the gluing of the parallelepiped faces is nontrivial, producing together with its geometry, this tessellation of Sol. The space looks compressed horizontally and vertically, depending on the sense. We highlight these regions with green dotted lines. In the bottom left, the camera points in the direction of x-axes, the group transformation in this direction is trivial. Thus we see copies of the fundamental domain distorted by Sol geometry, they surround the horizontal compressed region. On the bottom right side, a view towards the diagonal of the fundamental domain. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

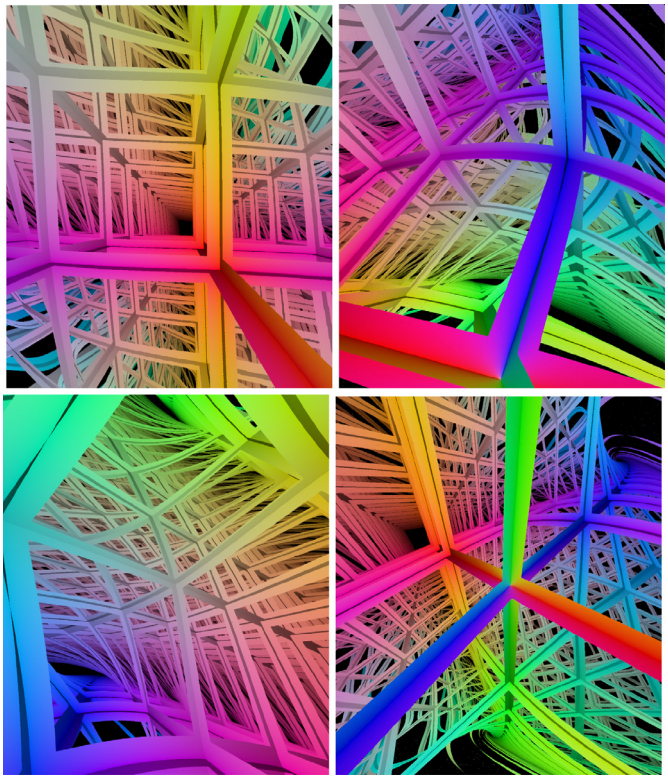


Fig. 8. Inside views of a parameterization of $SL_2(\mathbb{R})$. The scene is composed of a regular grid in $\mathbb{R}^3$ deformed by $SL_2(\mathbb{R})$ metric. The RGB colors are given by the x, y, z-coordinates of the hit points. In the top left, we set the camera towards the x-axes (the red line); this is a geodesic and coincides with a straight line in the Euclidean sense. In the top right, the camera points towards y-axes, which coincides with the green line. In the bottom left, the observer points at the z-direction (the blue edge). In the bottom right, the camera is aligned with the grid's diagonal. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

RGB color to be the (x, y, z) coordinates of the hit points. In future work, we intend to explore the rendering using the sectional curvature.

We illustrate four points of view in Fig. 8. To explain the images, we use the correspondence between $SL_2(\mathbb{R})$ and $\mathbb{H}^2 \times \mathbb{S}$ with an appropriate metric [3]. Here we consider the half-plane model of the two-dimensional hyperbolic geometry $\mathbb{H}^2$. In the top left, we look towards the x-axes (the red edge). This is a geodesic and coincides with a straight line in the Euclidean sense. It also matches with the line $(0, t, 0)$ in the model $\mathbb{H}^2 \times \mathbb{S}$. We avoid looking at the other sense because the parameterization that we are using is not defined for $x = 1$ (see Appendix C). In the top right, the camera points towards y-axes, which coincides with the green line. In the model $\mathbb{H}^2 \times \mathbb{S}$, it is the line $(t, 1, 0)$. In the bottom left, the observer is aligned to the z-axes (the blue edge). In the model $\mathbb{H}^2 \times \mathbb{S}$, this line is parameterized by $(t, t^2 + 1, \phi)$, where $\cos(\phi) = 1/\sqrt{t^2 + 1}$. Finally, in the bottom right, the camera points towards the diagonal.

8. Implementation details

In this section, we show how to map Algorithm 1 to the RTX pipeline and present the details of the GPU implementation.

8.1. RTX pipeline

NVidia RTX is a hardware/software platform with support for real-time ray tracing. The ray tracing code of an application using this architecture consists of CPU host code, GPU device code, and the memory to transfer data to the acceleration data structures for fast geometry culling when intersecting rays with scene objects. Specifically, the CPU host code manages the memory flow between devices, sets up, controls, and spawn GPU shaders and defines the Acceleration Structures.

These shaders correspond to tasks in Algorithm 1. The *Ray Generation Shader* is responsible for creating the rays (line 1 in Alg 1) defined by their origins and directions (line 2). A call to TraceRay() launches a ray (line 3). The next stage is the fixed traversal of the Acceleration Structure, controlled by the RTX runtime. This uses a custom *Intersection Shader* to compute intersections (line 5). In our case, we avoid it and use the built-in triangle intersection shader. All hits found are then sorted by the runtime and the closest hit is found. The *Closest-Hit Shader* is called for this intersection point (line 7). If no hit is found, the *Miss Shader* is called as a fallback. It is important to note that additional rays can be launched in the Closest-Hit/Miss shaders.

Scene objects and *fundamental domain faces* are treated differently when mapping Algorithm 1 to the RTX pipeline. The scene objects are tested and shaded in the regular way (lines 6-8), and the fundamental domain faces update the rays (lines 10-11). This is implemented with a custom-designed Miss Shader.

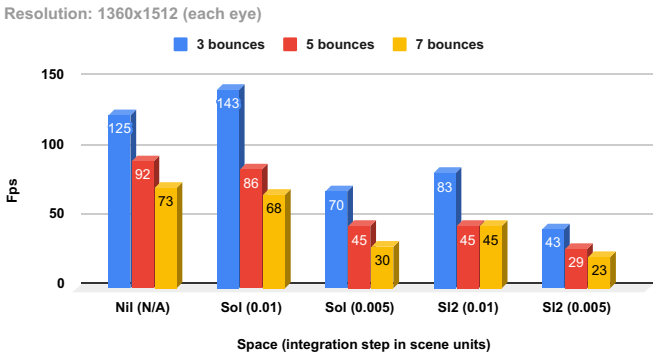


Fig. 9. Performance evaluation. We test two parameters: the number of ray bounces and the ray integration step size. Performance scales with number of bounces and the algorithm can be set to render all spaces at VR rates (near 90 fps). Resolution is 1360x1512 for each eye. Note that step size does not apply to Nil because we have used a closed form expression for the geodesics.

8.2. GPU implementation

The implementation of our visualization platform in GPU is built on top of NVIDIA's Falcor [26] real-time rendering framework using DirectX 12 (DXR) on Windows 10. Falcor development framework consists of a library with support for DXR at a high level and a built-in scene description format. We use the software Blender to create the scene objects.

The core functionality of our system's architecture consists of a set of shaders that are mapped to the RTX GPU pipeline as described above. We developed generic shaders for each stage of the GPU ray tracing pipeline that are independent of the geometric structure of the Non-Euclidean space.

Ray Generation Shader: Creates camera rays using the isometries of the space to transform the ray origin and direction to the camera coordinate system.

Intersection Shader:Uses the built-in intersection shader.

Closest Hit Shader: Does the Riemannian shading operation.

Miss Shader: Deals with the transport of rays in the fundamental domain. For this, the rays are tested for intersection with the boundary of the polyhedron Δ.

9. Experiments and comparisons

To evaluate the proposed model, we consider the implementation, discussed in previous section, using NVIDIA's Falcor [26]. Falcor provides a platform that supports many features for real-time visualization, including OpenVR and DirectX Raytracing. However, ray tracing and virtual reality do not work originally in an integrated way there. Thus, we relied on an integration extension [27]. The test system is a i7-8700 CPU at 3.20 GHz with 16GB RAM and a RTX 2080 Ti GPU.

The experimental methodology consists of varying the number of ray bounces and the step size for the numerical integration and evaluating how performance and display quality are affected. Fig. 9 shows the performance results. The image resolution for each eye is 1360×1512. It is important to emphasize that the scene is effectively ray-traced two times, one for each eye. The model succeeds in rendering high definition images in rates compatible with VR devices (90 fps) for Nil, Sol, and $SL_2(\mathbb{R})$, depending on proper parameter setup. The frame rate of $SL_2(\mathbb{R})$ is slower than that of Sol because it performs more computations in the shader since $SL_2(\mathbb{R})$ has more non-zero Christoffel symbols.

Given that performance is dependent on parameters, we evaluate how they affect the final image. Fig. 11 shows the results for Sol. As expected, the number of bounces controls how far we see the tessellation of the covering space by replication of the fundamental domain. Even a small value is useful for giving an intuition of the manifold. As expected, varying the integration step size is a trade-off between performance and precision.

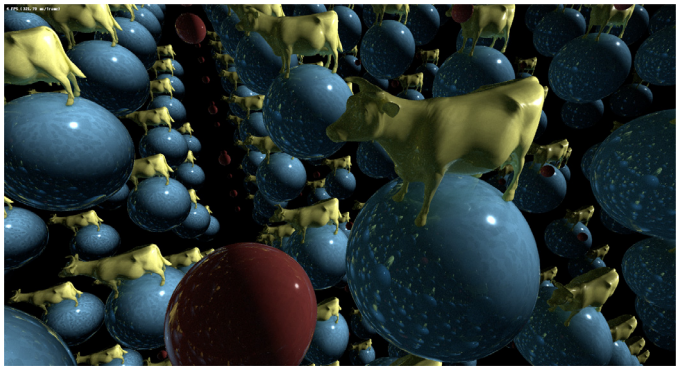


Fig. 10. Inside photorealistic view of the flat torus using an extension of the ray tracer algorithm [28]. The scene is composed of two spheres and a cow model.

10. Conclusion and future work

We introduced a real-time inside view of some three-dimensional manifolds based on ray tracing. This is the first project presenting (real-time) inside views of Nil, Sol, and $\widetilde{SL_2}(\mathbb{R})$ spaces using such technique. Previous works, such as Berger et al. [13] were restricted to Euclidean, hyperbolic and spherical spaces. The algorithm was based on Falcor, NVIDIA's scientific prototyping framework, which uses the power of the new generation of RTX GPUs.

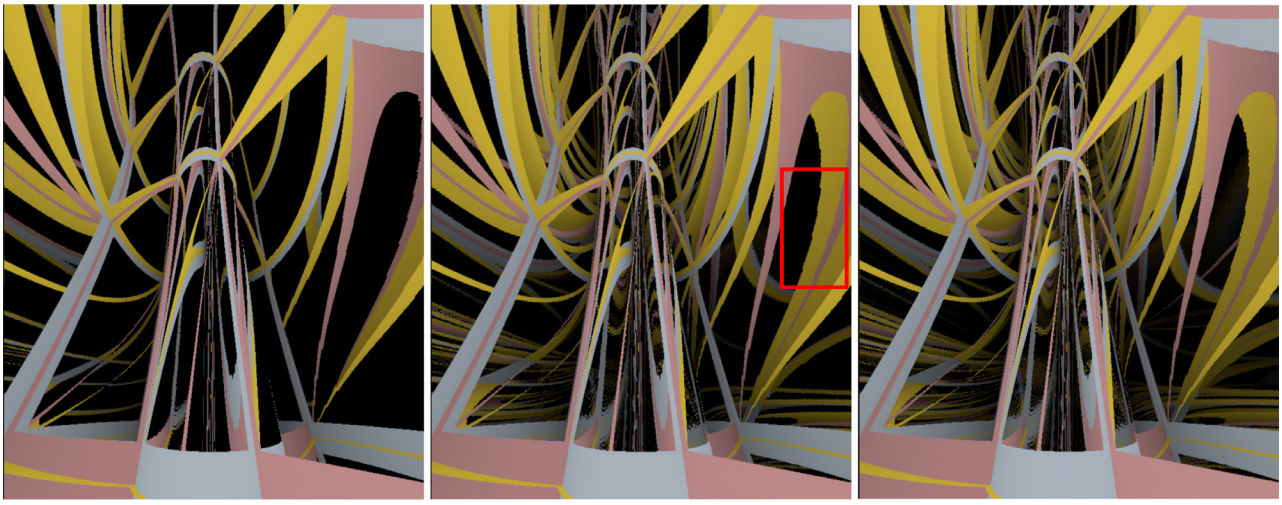
Our framework could be used to investigate abstract phenomena in low-dimensional geometry/topology. It also suggests new possibilities for the dissemination of Non-Euclidean spaces in games, art, and education.

It is worth mentioning that being inside a curved space is far more intuitive than seeing it on a display. For example, deformations are created when we rotate our head inside such spaces — this is due to their non-isotropic nature. Also, walking in the space using the Euclidean coordinates shows how different it is from the classic geometries. The Sol space, in particular, is surprising.

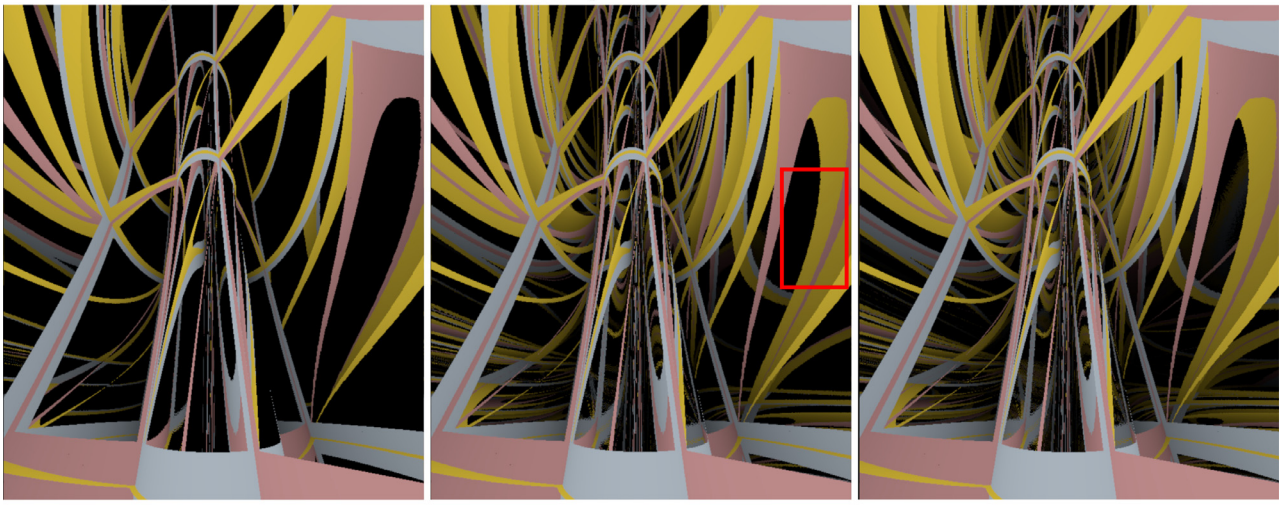
An interesting path for future work is to evaluate our algorithm through formal user experiments. This will help to understand the impact of immersive visualizations on people. Interesting questions include: does visualization help teaching the abstracts concepts related to the spaces? How insightful they can be to experts in the area? How inspiring can they be for artists?

We also intend to investigate global illumination of Non-Euclidean geometry. A recent attempt [28] to explore the Non-Euclidean illumination of the flat torus resulted in interesting images (see Fig. 10).

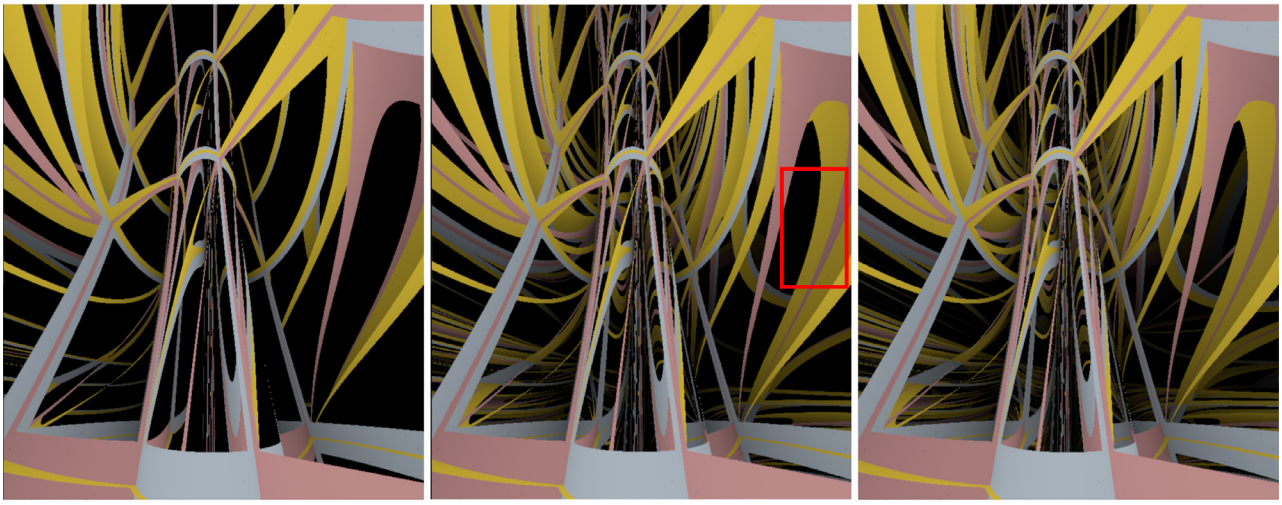
This paper deals with mathematical visualization, however, in this same line there is the interest of visualizing physical concepts. The four-dimensional *space-time* is one of these. Weiskopf [29] used ray tracing to explore space-time by simulating travels faster than the light speed. Gröller [30] visualized relativistic effects, the geometric behavior of nonlinear dynamical systems, and the movement of charged particles in a force field (e.g., electron movement). Additionally, the website [31] presents some interesting relativistic images. However, those visualizations are not in real-time. Approaching this problem using our framework could be an interesting challenge. Another one is the visualization of spaces related to *string theory* [32], especially the concepts of *D-branes* [33].



(a) Step: 0.01. Bounces: 3. (b) Step: 0.01; Bounces: 5. (c) Step: 0.01; Bounces: 7.



(d) Step: 0.005; Bounces: 3. (e) Step: 0.005; Bounces: 5. (f) Step: 0.005; Bounces: 7.



(g) Step: 0.001; Bounces: 3. (h) Step: 0.001; Bounces: 5. (i) Step: 0.001; Bounces: 7.

Fig. 11. Rendering evaluation of Sol manifold. The Figures show variations on the integration step (vertical) and number of bounces (horizontal). As bounces increase, more parallelepipeds tessellating space become visible. Even 3 bounces provide a good space intuition, as can be seen in (a), (d) and (g). The red rectangles in (b), (e) and (h) emphasize approximation improvements resulting from finer integration step. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Tiago Novello: Formal analysis, Conceptualization, Investigation, Methodology, Software, Writing - review & editing. **Vinícius da Silva:** Validation, Conceptualization, Investigation, Methodology, Software, Writing - review & editing. **Luiz Velho:** Supervision, Conceptualization, Investigation, Methodology, Software, Writing - review & editing.

Acknowledgments

We thank Ingo Wald for his idea (of future work) of using many segments of the polygonal $\{p_i\}$ to increase the GPU occupancy and optimize the intersection computations.

Appendix A. Nil space

We follow the definitions of Martelli [3]. *Nil space* is a Lie group consisting of all 3×3 real matrices

$$\begin{bmatrix} 1 & x & z \\ 0 & 1 & y \\ 0 & 0 & 1 \end{bmatrix}$$

with the multiplication operation. We identify $\mathbb{R}^3$ and *Nil* using

$$(x,y,z)\in\mathbb{R}^3 \to \begin{bmatrix} 1 & x & z \\ 0 & 1 & y \\ 0 & 0 & 1 \end{bmatrix} \in Nil$$

as a parameterization. For ray tracing, we set up our scene in $\mathbb{R}^3$. Then, the multiplication of (x,y,z) and (x',y',z') is:

$$\begin{aligned}(x,y,z)\cdot(x',y',z') &= \begin{bmatrix} 1 & x & z \\ 0 & 1 & y \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} 1 & x' & z' \\ 0 & 1 & y' \\ 0 & 0 & 1 \end{bmatrix} \\ &= (x+x', y+y', z+z'+xy').\end{aligned}$$

In other words, the multiplication of elements in *Nil* is the sum of its coordinates, with an additional term in the last one. This term makes all the difference, since, to put geometry in *Nil*, we consider the left multiplication $(x,y,z)\to p\cdot(x,y,z)$, for all $p\in Nil$, being isometry.

A.1. Nil metric

We construct a metric in Nil as in Section 4.4. The origin of *Nil* is $e=(0,0,0)$, and $T_eNil=\mathbb{R}^3$ is the tangent space at e with the Euclidean product $\langle u,v\rangle_e = u_xv_x+u_yv_y+u_zv_z$. Let p be a point in *Nil*, we define a scalar product $\langle\cdot,\cdot\rangle_p$ in T_pNil. Since $p^{-1}=(-p_x,-p_y,-p_z+p_xp_y)$ we have the isometry that translates p to the origin e:

$$\varphi(x,y,z) := L_{p^{-1}}(x,y,z) = (x-p_x, y-p_y, z-p_z+p_x(p_y-y)).$$

The differential of φ at a point p in the direction v is given by $d\varphi_p(v) = v_x\cdot d\varphi_p(e_1)+v_y\cdot d\varphi_p(e_2)+v_z\cdot d\varphi_p(e_3)$, where $\{e_1,e_2,e_3\}$ is standard base of $\mathbb{R}^3$. The partial derivatives are $d\varphi_p(e_1)=(1,0,0)$, $d\varphi_p(e_2)=(0,1,p_x)$, and $d\varphi_p(e_3)=(0,0,1)$, hence $d\varphi_p(v)=(v_x,v_y,v_yp_x+v_z)$. Eq. (5) gives the product between two directions u and v at p:

$$\langle u,v\rangle_p = \langle d\varphi_p u, d\varphi_p v\rangle_e = u^T\begin{bmatrix} 1 & 0 & 0 \\ 0 & p_x^2+1 & -p_x \\ 0 & -p_x & 1 \end{bmatrix} v.$$

The 3×3 matrix above defines a metric g_{ij} at p. Varying p we obtain a Riemannian metric $\langle\cdot,\cdot\rangle$, since each matrix entry is differentiable. The vectors (1,0,0), (0, 1, x), and (0,0,1) form an orthogonal basis at (x,y,z). Also, the volume form of *Nil* coincides with the standard one from $\mathbb{R}^3$, since $\det\left[g_{ij}\right]=1$.

Christoffel symbols in *Nil* at each point (x,y,z) are computed using the Formula (4). Their are all zero except for [3]:

$$\Gamma^1_{22} = -x,\ \Gamma^1_{23}=\Gamma^1_{32}=\frac{1}{2},$$
$$\Gamma^2_{12}=\Gamma^2_{21}=\frac{x}{2},\ \Gamma^2_{13}=\Gamma^1_{31}=-\frac{1}{2},$$
$$\Gamma^3_{12}=\Gamma^3_{21}=\frac{(x^2-1)}{2},\ \Gamma^3_{13}=\Gamma^3_{31}=-\frac{x}{2}.$$

The calculation can be done using the software *Maple*, for example [24]. Replacing the Christoffel symbols of *Nil* in Eq. (2) we obtain the geodesic flow of *Nil*.

A.2. Nil rays

The geodesic flow of *Nil* admits a solution [24]. A ray $\gamma(t)=(x(t),y(t),z(t))$ starting at (0,0,0) in the unit tangent direction $v=(c\cos(\alpha),c\sin(\alpha),w)$ has the following form:

$$\begin{aligned}x(t) &= \frac{c}{w}(\sin(wt+\alpha)-\sin(\alpha)) \\ y(t) &= -\frac{c}{w}(\cos(wt+\alpha)-\cos(\alpha)) \\ z(t) &= t(w+\frac{c^2}{2w})-\frac{c^2}{4w^2}(\sin(2wt+2\alpha)-\sin(2\alpha)) \\ &\quad+\frac{c^2}{2w^2}(\sin(wt+2\alpha)-\sin(2\alpha)-\sin(tw)).\end{aligned}$$

To compute a geodesic $\beta(t)$ starting at p in the direction v, we translate β to the origin using the left multiplication $\varphi(x,y,z)=(-p_x,-p_y,-p_z+p_xp_y)\cdot(x,y,z)$. Observe that, $\varphi(0)=e$ and $d\varphi_p(v)=v-(0,0,p_xv_y)$. Therefore, the curve $\varphi\circ\beta(t)=(x(t),y(t),z(t))$ is a ray starting at e in the direction $v-(0,0,p_xv_y)$, and it can be computed using the above closed formula. To compute $\beta(t)$ we simply apply L_p to $\varphi\circ\beta(t)$ to obtain the desired formula

$$\begin{aligned}\beta(t) &= p\cdot(x(t),y(t),z(t)) \\ &= (p_x+x(t), p_y+y(t), p_z+z(t)+p_xy(t)).\end{aligned}$$

Appendix B. Sol space

Sol is the least symmetric one among Thurston geometries. As in the Nil space, we follow the definitions of Martelli [3]. The *Sol space* is a Lie group consisting of all matrices

$$\begin{bmatrix} e^z & 0 & x \\ 0 & e^{-z} & y \\ 0 & 0 & 1 \end{bmatrix}$$

with the multiplication operation. *Sol* identifies with $\mathbb{R}^3$, since

$$(x,y,z)\in\mathbb{R}^3 \to \begin{bmatrix} e^z & 0 & x \\ 0 & e^{-z} & y \\ 0 & 0 & 1 \end{bmatrix} \in Sol$$

defines a parameterization. We push-forward the differentiable structure of $\mathbb{R}^3$ to *Sol* and use $\mathbb{R}^3$ to set our scene for ray tracing.

Let (x,y,z) and (x',y',z') be elements in *Sol*, their multiplication has the form:

$$\begin{aligned}(x,y,z)\cdot(x',y',z') &= \begin{bmatrix} e^z & 0 & x \\ 0 & e^{-z} & y \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} e^{z'} & 0 & x' \\ 0 & e^{-z'} & y' \\ 0 & 0 & 1 \end{bmatrix} \\ &= (x'e^z+x, y'e^{-z}+y, z+z').\end{aligned}$$

Specifically, the multiplication in *Sol* is the sum of the coordinates of the elements with an additional exponential term. To put a geometry in *Sol*, we follow Section 4.4 considering the left multiplication $(x, y, z) \to p \cdot (x, y, z)$ being a isometry.

B.1. Sol metric

The element $e = (0, 0, 0)$ is the identity of *Sol*. We consider the tangent space T_eSol at e to be endowed with the Euclidean scalar product. Let p be a point in *Sol*, we define a scalar product $\langle \cdot, \cdot \rangle_p$ in T_pSol as in Section 4.4. Then, we use the left multiplication $\varphi := L_{p^{-1}}$, and its differential $d\varphi_p$, to transpose vectors from T_pSol to T_eSol. As $p^{-1} = (-p_x e^{p_z}, -p_y e^{-p_z}, -p_z)$, we have $\varphi(x, y, z) = (xe^{-p_z} - p_x e^{p_z}, ye^{p_z} - p_y e^{-p_z}, z - p_z)$.

The differential of φ applied to a tangent vector v has the form $d\varphi_p(v) = v_x \cdot d\varphi_p(e_1) + v_y \cdot d\varphi_p(e_2) + v_z \cdot d\varphi_p(e_3)$. Computing the partial derivatives, $d\varphi_p(e1) = (e^{-p_z}, 0, 0)$, $d\varphi_p(e2) = (0, e^{p_z}, 0)$, and $d\varphi_p(e3) = (0, 0, 1)$, we obtain $d\varphi_p(v) = (v_x e^{-p_z}, v_y e^{p_z}, v_z)$.

Then, using Eq. (5), we derive the scalar product between two tangent vectors u and v at p as

$$\langle u, v \rangle_p = \langle d\varphi_p(u), d\varphi_p(v) \rangle_e = u^T \begin{bmatrix} e^{2p_z} & 0 & 0 \\ 0 & e^{-2p_z} & 0 \\ 0 & 0 & 1 \end{bmatrix} v.$$

The 3×3 matrix above defines a metric at p. Varying p we obtain a Riemannian metric $\langle \cdot, \cdot \rangle$, since each matrix entry is differentiable. The volume form of *Sol* coincides with the standard one from $\mathbb{R}^3$, since the determinant of the above matrix is one. To compute a ray in *Sol* we need the Christoffel symbols at each point (x, y, z), which are all zero except ([3]):

$$\Gamma^1_{13} = \Gamma^1_{31} = 1, \ \Gamma^2_{23} = \Gamma^2_{32} = -1,$$
$$\Gamma^3_{11} = -e^{2z}, \ \Gamma^3_{22} = \Gamma^3_{31} = e^{-2z}.$$

B.2. Sol rays

Replacing the Christoffel symbols of *Sol* in Eq. (3) we obtain the *geodesic flow*:

$$\begin{cases} x'_k = y_k, \ k = 1, 2, 3. \\ y'_1 = -2y_1 y_3 \\ y'_2 = 2y_2 y_3 \\ y'_3 = e^{2p_z} y_1^2 - e^{-2p_z} y_2^2 \end{cases} \tag{B.1}$$

Let $p \in Sol$ and $v \in T_pSol$ be an initial condition for Eq. (B.1). Then, there is no solution for this problem in terms of elementary functions [34]. To overcome this we use Euler's numerical method to integration as described in Section 6.3.

Appendix C. $\widetilde{SL_2}(\mathbb{R})$ space

We follow the definitions of Gilmore [35]. The *special linear group* $SL_2(\mathbb{R})$ consisting of all 2×2 matrices with unit determinant is a Lie group. Indeed, the product of two matrices with unit determinant has unit determinant, the same for the inverse matrix. To understand the richness of $SL_2(\mathbb{R})$, we present two interpretations: one geometrical and another more topological.

Geometrical interpretation:

We can write $SL_2(\mathbb{R}) = \{(a, b, c, d) \in \mathbb{R}^4 | \ ad - bc = 1\}$, that is, a 3-manifold in $\mathbb{R}^4$. Moreover, rephrasing $ad - bc = 1$ we obtain:

$$\left(\frac{a+d}{2}\right)^2 - \left(\frac{a-d}{2}\right)^2 + \left(\frac{b-c}{2}\right)^2 - \left(\frac{b+c}{2}\right)^2 = 1,$$

which describes the equation of a 3-hyperbola in $\mathbb{R}^4$.

Topological interpretation: We can also identify $SL_2(\mathbb{R})$ with the product of the (hyperbolic) *upper half plane* $\mathbb{H} = \{z \in \mathbb{C} | \ Im(z) > 0\}$ and the unit circle $\mathbb{S}^1$. Before, we provide some additional definitions. An element $g = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \in SL_2(\mathbb{R})$ induces an *action* in $\mathbb{H}$:

$$gz = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} z \\ 1 \end{bmatrix} = \begin{bmatrix} az + b \\ cz + d \end{bmatrix} = \frac{az + b}{cz + d}. \tag{C.1}$$

We used homogeneous coordinates in Eq. (C.1).

The action in Eq. (C.1) is well defined, that is $cz + d \neq 0$ and $Im(gz) > 0$. As $ad - bc = 1$, either c or d are non null, so is $cz + d$. To verify that the imaginary part of gz is greater than zero we multiply the numerator and denominator of gz by $c\bar{z} + d$, where $\bar{z}$ is the conjugate of z. After some calculations we get $Im(gz) = Im(z)/|cz + d|^2$, which is greater than zero.

We now provide a decomposition of g in two components AN and K, known as *Iwasawa decomposition*. Where K will be responsible by rotations around the point $i \in \mathbb{H}^2$ and AN will be translations of i. This procedure provides the decomposition of $SL_2(\mathbb{R})$ into the product $\mathbb{S}^1 \times \mathbb{H}^2$. Specifically, define

$$AN = \begin{bmatrix} \frac{1}{\sqrt{c^2+d^2}} & \frac{ac+bd}{\sqrt{c^2+d^2}} \\ 0 & \sqrt{c^2+d^2} \end{bmatrix}, \ K = \begin{bmatrix} d & -c \\ c & d \end{bmatrix} \cdot \frac{1}{\sqrt{c^2+d^2}}.$$

It is possible to verify that the product $AN \cdot K$ is g. Observe that K can be written as $\begin{bmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{bmatrix}$, for some angle $0 \leq \phi \leq 2\pi$. Thus, the matrices K can be parameterized by $\mathbb{S}^1$. We now verify that AN parameterize $\mathbb{H}^2$.

Let $z = x + iy$ be a point in $\mathbb{H}^2$, then

$$z = \begin{bmatrix} \sqrt{y} & \frac{x}{\sqrt{y}} \\ 0 & \frac{1}{\sqrt{y}} \end{bmatrix} \begin{bmatrix} i \\ 1 \end{bmatrix} = x + iy. \tag{C.2}$$

Every point in $\mathbb{H}^2$ can be written as the action of some matrix in $SL_2(\mathbb{R})$ on i. Taking $y = c^2 + d^2$ and $x = ac + bd$ the correspondence between AN and $\mathbb{H}^2$ follows.

The identification of $SL_2(\mathbb{R})$ with $\mathbb{H}^2 \times \mathbb{S}^1$ provides a topological interpretation of $SL_2(\mathbb{R})$: its fundamental group is $\mathbb{S}^1$. Then $SL_2(\mathbb{R})$ is not simply connected, hence it is not a model geometry. However, its *universal cover* $\widetilde{SL_2(\mathbb{R})}$ is a model geometry[21]. We focus on $SL_2(\mathbb{R})$ since they have identical geometry and we are dealing with visualization.

We could use the above parameterization of $SL_2(\mathbb{R})$ by $\mathbb{H}^2 \times \mathbb{S}^1$, however, we take an easier coordinate system. We parameterize a neighborhood of the identity of $SL_2(\mathbb{R})$ by a neighborhood of the origin of $\mathbb{R}^3$ using the map [35]:

$$X(x, y, z) = \begin{bmatrix} 1 + x & y \\ z & \frac{1 + yz}{1 + x} \end{bmatrix}. \tag{C.3}$$

Observe that $X(0, 0, 0)$ is the identity of $SL_2(\mathbb{R})$, and that in the plane $x = 1$ the map is not defined. We use X to push-back the metric of $SL_2(\mathbb{R})$ to $\mathbb{R}^3$.

C.1. $SL_2(\mathbb{R})$ metric

We construct a metric in the $SL_2(\mathbb{R})$ and use the map defined in Eq. (C.3) to push it to $\mathbb{R}^3$.

The identity matrix is the identity e of $SL_2(\mathbb{R})$. Let $T_eSL_2(\mathbb{R})$ be the tangent space at e with the well known [35] matrix scalar product $\langle u, v \rangle_e = Tr(u \cdot v)$ between two 2×2 matrices u and v in $T_eSL_2(\mathbb{R})$. Tr is the matrix trace. As in Section 4.4, we define a

scalar product $\langle \cdot, \cdot \rangle_p$ in $T_p SL_2(\mathbb{R})$ for a point p using the left multiplication φ, where

$$p^{-1} = \begin{bmatrix} \frac{1+p_y p_z}{1+p_x} & -p_y \\ -p_z & 1+p_x \end{bmatrix}.$$

Then we obtain the parameterization $\varphi(x,y,z) = p^{-1} \cdot X(x,y,z)$. Its differential at p, applied to the vector v, has the form $d\varphi_p(v) = v_x \cdot d\varphi_p(e_1) + v_y \cdot d\varphi_p(e_2) + v_z \cdot d\varphi_p(e_3)$. The partial derivatives are:

$$d\varphi_p(e1) = \begin{bmatrix} \frac{1+p_y p_z}{1+p_x} & \frac{p_y + p_y^2 p_z}{(1+p_x)^2} \\ -p_z & -\frac{1+p_y p_z}{1+p_x} \end{bmatrix},$$

$$d\varphi_p(e2) = \begin{bmatrix} 0 & \frac{1}{1+p_x} \\ 0 & 0 \end{bmatrix}, \quad d\varphi_p(e3) = \begin{bmatrix} -p_y & -\frac{p_y^2}{1+p_x} \\ -1+p_x & p_y \end{bmatrix}.$$

After some computations using the scalar product at $T_e SL_2(\mathbb{R})$ we obtain the metric tensor at p:

$$\begin{bmatrix} 2\frac{(1+p_y p_z)}{(1+p_x)^2} & -\frac{p_z}{1+p_x} & -\frac{p_y}{1+p_x} \\ -\frac{p_z}{1+p_x} & 0 & 1 \\ -\frac{p_y}{1+p_x} & 1 & 0 \end{bmatrix}. \tag{C.4}$$

C.2. $SL_2(\mathbb{R})$ rays

To compute a ray in $SL_2(\mathbb{R})$ we need the Christoffel symbols at each point $\varphi(p)$, which are all zero except for

$$\Gamma^1_{11} = -\frac{1+p_y p_z}{1+p_x}, \ \Gamma^1_{12} = \frac{p_z}{2}, \ \Gamma^1_{13} = \frac{p_y}{2}, \ \Gamma^1_{23} = -\frac{1+p_x}{2}$$

$$\Gamma^2_{11} = -\frac{p_y + p_y^2 p_z}{(1+p_x)^2}, \ \Gamma^2_{12} = \frac{p_y p_z}{2+2p_x}, \ \Gamma^2_{13} = \frac{p_y^2}{2+2p_x}, \ \Gamma^2_{23} = -\frac{p_y}{2}$$

$$\Gamma^3_{11} = -\frac{p_z + p_z^2 p_y}{(1+p_x)^2}, \ \Gamma^3_{12} = \frac{p_z^2}{2+2p_x}, \ \Gamma^3_{13} = \frac{p_y p_z}{2+2p_x}, \ \Gamma^3_{23} = -\frac{p_z}{2}.$$

Remember that $\Gamma^k_{ij} = \Gamma^k_{ji}$. We obtain the geodesic flow by replacing the Christoffel symbols in Eq. (3).

$$\begin{cases} x'_k = y_k, \ k = 1,2,3. \\ y'_1 = \frac{(1+p_y p_z) y_1^2}{1+p_x} - p_z y_1 y_2 - p_y y_1 y_3 + (1+p_x) y_2 y_3 \\ y'_2 = \frac{(1+p_y p_z) p_y y_1^2}{(1+p_x)^2} - \frac{p_z p_y}{1+p_x} y_1 y_2 - \frac{p_y^2}{1+p_x} y_1 y_3 + p_y y_2 y_3 \\ y'_3 = \frac{(1+p_y p_z) p_z y_1^2}{(1+p_x)^2} - \frac{p_z^2}{1+p_x} y_1 y_2 - \frac{p_y p_z}{1+p_x} y_1 y_3 + p_z y_2 y_3 \end{cases}$$

We use Euler's numerical method to integrate this equation.

Supplementary material

Supplementary material associated with this article can be found, in the online version, at doi:10.1016/j.cag.2020.07.016.

References

[1] Gegenberg J, Vaidya S, Vazquez-Poritz JF. Thurston geometries from eleven dimensions. Class Quant Grav 2002;19(23):L199.
[2] Weeks J. The shape of space. Marcel Dekker; 2002a. ISBN 9780824707095.
[3] Martelli B. 2016. An introduction to geometric topology. arXiv preprint arXiv:1610.02592.
[4] Amenta N, Levy S, Munzner T, Phillips M. Geomview: a system for geometric visualization. In: Proceedings of the eleventh annual symposium on Computational geometry. ACM; 1995. p. 412–13.
[5] Weeks J. Real-time rendering in curved spaces. IEEE Comput Graph Appl 2002b;22(6):90–9.
[6] Weeks J. Real-time animation in hyperbolic, spherical, and product geometries. In: Non-euclidean geometries. In: Mathematics and Its Applications, 581. Springer US; 2006. p. 287–305.
[7] Hudson R, Gunn C, Francis GK, Sandin DJ, DeFanti TA. Mathenautics: using vr to visit 3-d manifolds. In: Proceedings of the 1995 symposium on Interactive 3D graphics. New York, NY, USA: ACM; 1995. p. 167–70. ISBN 0-89791-736-7.
[8] Francis GK, Goudeseune CMA, Kaczmarski HJ, Schaeffer BJ, Sullivan JM. Alice on the eightfold way: exploring curved spaces in an enclosed virtual reality theatre. In: Visualization and mathematics III; 2003. p. 305–15.
[9] Hart V, Hawksley A, Matsumoto EA, Segerman H. Non-euclidean virtual reality i: explorations of h^3. In: Proceedings of bridges 2017: mathematics, music, art, architecture, culture; 2017.
[10] Weeks J. Non-euclidean billiards in vr. In: Proceedings of bridges 2020: mathematics, music, art, architecture, culture.
[11] Weeks J. Virtual reality simulations of curved spaces. preparation
[12] Weismann S, Gunn C, Brinkmann P, Hoffmann T, Pinkall U. jreality: a java library for real-time interactive 3d graphics and audio.. In: ACM multimedia'09; 2009. p. 927–8.
[13] Berger P, Laier A, Velho L. An image-space algorithm for immersive views in 3-manifolds and orbifolds. Vis Comput 2015;31(1):93–104.
[14] Velho L, Novello T, Silva V, Lucio D. Visualization of non-euclidean spaces using ray tracing. Technical Report. VISGRAF Lab - IMPA; 2019.
[15] Velho L, da Silva V, Novello T. Immersive visualization of the classical non-euclidean spaces using real-time ray tracing in vr. In: Proceedings of the 46th graphics interface; 2020.
[16] Anonymous. Visualization of nil, sl2, and sol. 2019. Available as additional material.
[17] Coulon R., Matsumoto E.A., Segerman H., Trettel S. 2020. Non-euclidean virtual reality III: Nil. arXiv preprint arXiv:2002.00513.
[18] Coulon R., Matsumoto E.A., Segerman H., Trettel S. 2020. Non-euclidean virtual reality iv: Sol. arXiv preprint arXiv:2002.00369.
[19] Kopczyński E., Celińska-Kopczyńska D. 2020. Real-time visualization in non-isotropic geometries. arXiv preprint arXiv:2002.09533.
[20] Whitted T. An improved illumination model for shaded display. In: ACM SIGGRAPH Computer Graphics, 13. ACM; 1979. p. 14.
[21] Thurston W. The geometry and topology of three-manifolds. Princeton University; 1979.
[22] Carmo MPd. Riemannian geometry. Birkhäuser; 1992.
[23] Thurston WP. How to see 3-manifolds. Class Quant Grav 1998;15(9):2545.
[24] Szilágyi B, Virosztek D. Curvature and torsion of geodesics in three homogeneous riemannian 3-geometries. Stud Univ Žilina, Math Ser 2003;16:1–7.
[25] Thurston WP. Three-dimensional geometry and topology, 35. Princeton university press; 1997.
[26] Benty N., Yao K.-H., Foley T., Oakes M., Lavelle C., Wyman C.. The Falcor rendering framework. 2018. Https://github.com/NVIDIAGameWorks/Falcor.
[27] Silva V, Velho L. Ray tracing virtual reality in falcor : Ray-vr. Technical Report. VISGRAF Lab - IMPA; 2019.
[28] Novello T., da Silva V., Velho L. 2020. Global illumination of non-euclidean spaces. arXiv preprint arXiv:2003.11133.
[29] Weiskopf D. Visualization of four-dimensional spacetimes; 2001.
[30] Gröller E. Nonlinear ray tracing: visualizing strange worlds. Vis Comput 1995;11(5):263–74.
[31] Cicconet M.. VisualizaʌAo relativḍstica usando ray tracing. 2007. URL http://lvelho.impa.br/i3d07/demos/cicconet.
[32] Klimenko S, Nikitin I, Burkin V, Semenov V, Tarlapan O, Hagen H. Visualization in string theory. Comput Graph 2000;24:23–30.
[33] Johnson CV. D-branes. Cambridge university press; 2002.
[34] Szilágyi ABB. Frenet formulas and geodesics in sol geometry. Contrib Algebra Geom 2007;48(2):411–21.
[35] Gilmore R. Lie groups, physics, and geometry: an introduction for physicists, engineers and chemists. Cambridge University Press; 2008.